torch.nn.utils.parametrize.register_paran

https://pytorch.org/docs/stable/generated/torch.nn.utils.parametrize.registe

Description:

Register a parametrization to a tensor in a module. Assume that tensor_name="weight" for simplicity. When accessing module.weight, the module will return the parametrized version parametrization(module.weight). If the original tensor requires a gradient, the backward pass will differentiate through parametrization, and the optimizer will update the tensor accordingly. The first time that a module registers a parametrization, this function will add an attribute parametrizations to the module of type ParametrizationList. The list of parametrizations on the tensor weight will be accessible under module.parametrizations.weight. The original tensor will be accessible under module.parametrizations.weight.original.

Function Signature

```
torch.nn.utils.parametrize.register_parametrization(module,
tensor_name, parametrization, *, unsafe=False)[source]#
Parameters
Keyword Arguments
Raises
Return type
```

Parameters

Keyword

unsafe (bool) – a boolean flag that denotes whether the parametrization may change the dtype and shape of the tensor. Default: False Warning: the parametrization is not checked for consistency upon registration. Enable this flag at your own risk.

Returns:

Module

Code Examples

```
def right_inverse(self, X: Tensor) -> Union[Tensor,
Sequence[Tensor]]
```

```
>>> import torch
>>> import torch.nn as nn
>>> import torch.nn.utils.parametrize as P
>>>
>>> class Symmetric(nn.Module):
>>>     def forward(self, X):
>>>         return X.triu() + X.triu(1).T  # Return a symmetric
matrix
>>>
>>>     def right_inverse(self, A):
>>>         return A.triu()
>>>
>>> m = nn.Linear(5, 5)
>>> P.register_parametrization(m, "weight", Symmetric())
>>> print(torch.allclose(m.weight, m.weight.T))  # m.weight is
now symmetric
True
>>> A = torch.rand(5, 5)
>>> A = A + A.T  # A is now symmetric
>>> m.weight = A  # Initialize the weight to be the symmetric
matrix A
>>> print(torch.allclose(m.weight, A))
True
```

```
>>> class RankOne(nn.Module):
>>>     def forward(self, x, y):
>>> # Form a rank 1 matrix multiplying two vectors
>>>         return x.unsqueeze(-1) @ y.unsqueeze(-2)
>>>
>>>     def right_inverse(self, Z):
>>> # Project Z onto the rank 1 matrices
>>>         U, S, Vh = torch.linalg.svd(Z, full_matrices=False)
>>> # Return rescaled singular vectors
>>>         s0_sqrt = S[0].sqrt().unsqueeze(-1)
>>>         return U[..., :, 0] * s0_sqrt, Vh[..., 0, :] *
s0_sqrt
>>>
>>> linear_rank_one = P.register_parametrization(
...     nn.Linear(4, 4), "weight", RankOne()
... )
>>>
print(torch.linalg.matrix_rank(linear_rank_one.weight).item())
1
```

Notes & Warnings

NOTE Note If unsafe=False (default) both the forward and right_inverse methods will be called once to perform a number of consistency checks. If unsafe=True, then right_inverse will be called if the tensor is not parametrized, and nothing will be called otherwise.

NOTE Note In most situations, right_inverse will be a function such that forward(right_inverse(X)) == X (see right inverse). Sometimes, when the parametrization is not surjective, it may be reasonable to relax this.

⚠ WARNING Warning If a parametrization depends on several inputs, register_parametrization() will register a number of new parameters. If such parametrization is registered after the optimizer is created, these new parameters will need to be added manually to the optimizer. See torch.Optimizer.add_param_group().

Detailed Documentation

Documentation

Register a parametrization to a tensor in a module.

Assume that tensor_name="weight" for simplicity. When accessing module.weight, the module will return the parametrized version parametrization(module.weight). If the original tensor requires a gradient, the backward pass will differentiate through parametrization, and the optimizer will update the tensor accordingly.

The first time that a module registers a parametrization, this function will add an attribute parametrizations to the module of type ParametrizationList.

The list of parametrizations on the tensor weight will be accessible under module.parametrizations.weight.

The original tensor will be accessible under module.parametrizations.weight.original.

Parametrizations may be concatenated by registering several parametrizations on the same attribute.

The training mode of a registered parametrization is updated on registration to match the training mode of the host module

Parametrized parameters and buffers have an inbuilt caching system that can be activated using the context manager cached().

A parametrization may optionally implement a method with signature

This method is called on the unparametrized tensor when the first parametrization is registered to compute the initial value of the original tensor. If this method is not implemented, the original tensor will be just the unparametrized tensor.

If all the parametrizations registered on a tensor implement right_inverse it is possible to initialize a parametrized tensor by assigning to it, as shown in the example below.

It is possible for the first parametrization to depend on several inputs. This may be

implemented returning a tuple of tensors from right_inverse (see the example implementation of a RankOne parametrization below).

In this case, the unconstrained tensors are also located under module.parametrizations.weight with names original0, original1,…

If unsafe=False (default) both the forward and right_inverse methods will be called once to perform a number of consistency checks. If unsafe=True, then right_inverse will be called if the tensor is not parametrized, and nothing will be called otherwise.

In most situations, right_inverse will be a function such that forward(right_inverse(X)) == X (see right inverse). Sometimes, when the parametrization is not surjective, it may be reasonable to relax this.

If a parametrization depends on several inputs, register_parametrization() will register a number of new parameters. If such parametrization is registered after the optimizer is created, these new parameters will need to be added manually to the optimizer. See torch.Optimizer.add_param_group().

module (nn.Module) – module on which to register the parametrization

tensor_name (str) – name of the parameter or buffer on which to register the parametrization

parametrization (nn.Module) – the parametrization to register

unsafe (bool) – a boolean flag that denotes whether the parametrization may change the dtype and shape of the tensor. Default: False Warning: the parametrization is not checked for consistency upon registration. Enable this flag at your own risk.

ValueError – if the module does not have a parameter or a buffer named tensor_name